Practice Assignment – 1

(Covering Experiments 1–5)

F.Y. B.Tech (Common to All) | Academic Year 2026-27

Instructions

1. This assignment has **5 sections (A–E)**, each containing **10 questions**.
2. Every student must solve **exactly ONE question from each section** (5 questions in total), as allocated by the **last digit of your roll number** in the table below.
3. Section A corresponds to Experiment 1, Section B to Experiment 2, Section C to Experiment 3, Section D to Experiment 4, and Section E to Experiment 5.
4. Write the complete C program for each assigned question, along with the algorithm and/or flowchart, and the sample output obtained on execution.
5. All programs must be written and tested using only the constructs covered up to the corresponding experiment — do not use functions, structures, pointers, or recursion unless a question explicitly permits a specific library function.
6. Submit the assignment as per the schedule communicated by your faculty in-charge. A viva-voce may be conducted based on the submitted programs.

Question Allocation Based on Roll Number

Last Digit of Roll No.	Section A	Section B	Section C	Section D	Section E
1	Q1	Q11	Q21	Q31	Q41
2	Q2	Q12	Q22	Q32	Q42
3	Q3	Q13	Q23	Q33	Q43
4	Q4	Q14	Q24	Q34	Q44
5	Q5	Q15	Q25	Q35	Q45
6	Q6	Q16	Q26	Q36	Q46
7	Q7	Q17	Q27	Q37	Q47
8	Q8	Q18	Q28	Q38	Q48
9	Q9	Q19	Q29	Q39	Q49
0	Q10	Q20	Q30	Q40	Q50

Section A — Data Types & Operators

(based on Experiment 1)

1. Given two integers `a` and `b`, evaluate and print the following expressions, applying bitwise, logical, arithmetic, and comma operators:
```
result1 = a & b | (a ^ b) && (a << 2);
result2 = (a > b ?  a :  b) + (~a + 1) * b / 2;
result3 = (++a, b += 2, a > b ?  a :  b);
```

2. Given three integers `a, b, c` and a floating-point number `d`, evaluate the following expressions step by step, printing all intermediate results:
```
result1 = (a + b) * c / d - (b % a) + (c++);
result2 = (a > b && c < d) || (a + c > b * 2);
result3 = (a & b) ^ (c | ~b) ?  (a >> 1) :  (b << 1);
```

3. Given `qui = 4, add = 2, god = 2` (all `int`), evaluate the expression below and show the **stepwise hierarchy** of operations, similar to a trace table:
```
s = qui * add / 4 - 6 / 2 + 2 / 3 * 6 / god;
```

4. Given `a = 4, g = 3` (`int`), evaluate the expression below and show the stepwise hierarchy of operations:
```
s = 1 / 3 * a / 4 - 6 / 2 + 2 / 3 * 6 / g;
```

5. Accept three integers from the user. Using **only** the ternary, relational, and logical operators (no `if` statements), determine and print: (a) whether each integer is even or odd, and (b) whether all three are equal, exactly two are equal, or all three are distinct.

6. Ramesh's basic salary is entered through the keyboard. His dearness allowance is 40% of the basic salary, and house rent allowance is 20% of the basic salary. Write a program to calculate and display his gross salary.

7. A cashier has currency notes of denominations Rs. 100, Rs. 50, and Rs. 10 only. If the amount to be withdrawn (a multiple of 10, entered in hundreds) is given, compute the number of notes of **each** denomination required.

8. Accept the number of days from the user and convert it into years, weeks, and remaining days (ignore leap years), using only arithmetic and modulus operators.

9. Write a program that uses the `sizeof` operator to display the number of bytes occupied by each fundamental C data type (`int`, `float`, `double`, `char`), and at least one qualified type (e.g., `long int` or `unsigned int`).

10. Write a program that adds two `float` values and stores the result in a variable of type `int`. Print both the original float sum and the truncated int result, and explain in a comment which C type-conversion rule causes the truncation.

Section B — Decision Making & Branching

(based on Experiment 2)

11. Write a program to read the age of a person and determine if they are eligible to cast their vote. If the entered age is negative, display `"Invalid age entered"` instead of an eligibility result.

12. Write a program that checks if a positive integer is divisible by 3, by 7, by both, or by neither, and prints an appropriate message using logical operators combined with `if-else`.

13. Write a program to read the temperature in degrees Celsius and display a suitable weather message using an `else-if` ladder: below 0 → Freezing, 0–10 → Very Cold, 10–20 → Cold, 20–30 → Normal, 30–40 → Hot, 40 and above → Very Hot.

14. In a company, a worker's efficiency is judged by the time taken to complete a task: 2–3 hrs → Highly Efficient, 3–4 hrs → Improve Speed, 4–5 hrs → Needs Training, more than 5 hrs → Has to Leave. Accept the time taken and print the worker's efficiency category using an `else-if` ladder.

15. Write a program to compute a PWM duty cycle using nested conditions: if `Temp >= 50` AND `Switch == 1`, then if `Mode == 1` (automatic) set `PWM_Duty = 75`, otherwise set it to `50`; if the first condition is not met, set `PWM_Duty = 25`. Implement using nested `if-else` only.

16. Accept an integer `n`. If `n` is divisible by both 3 and 5, print `"RadhaKrishna"`. If divisible by 3 only, print `"Radha"`. If divisible by 5 only, print `"Krishna"`. Otherwise, print `"Neither"`. Use nested `if-else` only.

17. An electricity board charges as follows: first 100 units at Rs. 1.50/unit, the next 200 units (101–300) at Rs. 2.00/unit, and any units beyond 300 at Rs. 3.00/unit. Accept the total units consumed and compute the total bill using an `else-if` ladder.

18. A parking facility charges by vehicle type: Truck/Bus – Rs. 20/hr, Car – Rs. 10/hr, Scooter/Motorcycle – Rs. 5/hr. Accept the vehicle type (as a character) and the number of hours parked, and compute the total charge using `switch-case`.

19. Write a menu-driven program (using `switch-case`) for a simplified ATM that lets the user select **one** of: Check Balance, Deposit, or Withdraw, for a single transaction. Assume a fixed starting balance. For a withdrawal, use an `if-else` check to ensure the amount does not exceed the balance.

20. Write a program using `switch-case` to select a washing machine mode: 1: Quick Wash (25 min), 2: Deep Clean (45 min), 3: Spin Dry (15 min), 4: Rinse (10 min). For any other input, display `"Invalid mode selected."`

Section C — Looping Control Structures

(based on Experiment 3)

21. Write a program to print the following pattern for `n` rows, using nested loops:

```
1
1 2
1 2 3
1 2 3 4
```

22. Write a program to print the following mixed number/letter pyramid for a given number of rows, using nested loops:

```
1
B B
2 3 4
D D D D
5 6 7 8 9
```

23. Write a program to print the following diamond pattern of stars for a given odd value of `n`, using nested loops:

```
  *
 ***
*****
 ***
  *
```

24. Write a program to print the following pattern for `n` rows, where each row repeats its row number that many times, using nested loops:

```
1
22
333
4444
```

25. Write a program to print the following pattern of consecutive numbers for `n` rows, using nested loops:

```
1
2 3
4 5 6
7 8 9 10
```

26. Write a program to find and display all prime numbers between 1 and 300, using nested loops with `break` and `continue` used appropriately.

27. Write a program to check whether a given number is a **Krishnamurthy number** (a number whose sum of the factorial of its digits equals the number itself; e.g., $145 = 1! + 4! + 5!$).

28. Write a program to check whether a given positive integer is a **Perfect Number** (a number equal to the sum of its proper divisors, excluding itself; e.g., $6 = 1 + 2 + 3$).

29. Write a number-guessing game: the program stores a fixed secret number between 1 and 100, and the user has 7 attempts to guess it. After each guess, display `"Too High"`, `"Too Low"`, or `"Correct!"`, using a loop that ends when the number is guessed or the attempts run out.

30. Simulate a simple vending machine: the user selects a product (fixed price) and repeatedly enters payment in multiples of Rs. 5 (using a loop) until the total paid is at least the price; then display the change to be returned, if any.

Section D — Arrays

(based on Experiment 4)

31. Write a program to accept `n` elements into an array, sort them, and display the median of the sorted array (handle both odd and even `n`).

32. Write a program to generate an $n \times n$ square matrix ($2 \leq n \leq 10$) using nested loops, where: diagonal elements = square of their 1-based index; elements above the diagonal = $i + j$ (0-based indices); elements below the diagonal = $i \times j$. Replace any value divisible by 7 with -1, then display the matrix and count how many -1 values it contains.

33. Write a program to rotate the elements of a 1D array to the **left** by `k` positions (e.g., {1,2,3,4,5} rotated left by 2 becomes {3,4,5,1,2}).

34. Write a program to find two elements `arr[i]` and `arr[j]` (with `i < j`) in an array such that `arr[j] - arr[i]` is the maximum possible, and display this maximum difference along with the two elements.

35. Write a program to accept 25 numbers into an array and a number to search for. If found, display how many times it occurs in the array; otherwise, display an appropriate message.

36. Write a program to compute the **determinant** of a 3×3 matrix using the standard cofactor expansion formula, and display the result.

37. In a company, 5 salesmen each sell 3 products (A, B, C). Using a 2D array, write a program to display (a) the total sales by **each salesman**, and (b) the total sales of **each product**.

38. Write a program to multiply two 2×2 matrices and display the resulting matrix.

39. Write a program to accept an $N \times M$ 2D array and display its elements in **clockwise spiral order**.

40. Write a program to store the marks of 10 students in 3 subjects using a 2D array. Compute and display each student's total and average marks, storing each student's average in a **separate 1D array**.

Section E — Strings & String Handling Functions

(based on Experiment 5)

41. Write a program that accepts a string and computes the sum of the ASCII values of all its characters.

42. Write a program that accepts a binary number as a string (e.g., `"1011"`) and converts it to its decimal equivalent.

43. Write a program that accepts a string, a word, and a position, and inserts the word into the string at the given position (e.g., `"Fun to learn C Programming"` with word `"PIC"` at position 14 becomes `"Fun to learn PIC C Programming"`).

44. Write a program that accepts a string and two positions `n1` and `n2`, and reverses only the portion of the string between those two positions, leaving the rest unchanged.

45. Write a program to compare two strings character by character **without using any built-in string library function**, and report whether they are equal.

46. Write a program to concatenate two strings manually using array indexing, **without using `strcpy()` or `strcat()`**.

47. Write a program to accept a list of `n` names (as an array of strings) and sort them into alphabetical order.

48. Write a program that accepts a sentence (at most 100 characters) and: (a) counts the frequency of each vowel (case-insensitive), (b) identifies the most frequently occurring character, (c) replaces every vowel with the next vowel in cyclic order ($a \to e \to i \to o \to u \to a$), and (d) reverses the characters of **each word** while preserving the order of the words.

49. Write a program to check whether a given identifier name follows valid C naming rules: it must start with a letter or underscore, contain only letters/digits/ underscores, and must not match any name in a small fixed list of C reserved keywords.

50. Write a program that accepts a sentence and, **without using any built-in string function**, counts and displays the number of vowels, consonants, digits, and spaces present in it.